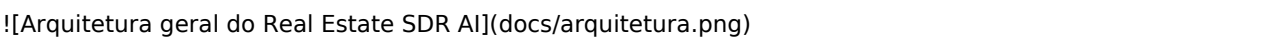
Real Estate SDR --- Arquitetura e Guia de Execução

Arquitetura da Solução



Documentação dos dois serviços da solução:

- `real-estate-sdr-api`: backend Go, API REST e persistência PostgreSQL.
- `real-estate-sdr-ai`: serviço Python/FastAPI para atendimento conversacional, qualificação do lead, integração com LLM e recomendação de imóveis.

> Os repositórios são separados. Este documento considera Windows PowerShell e os caminhos usados no desenvolvimento. Confirme nomes de variáveis, rotas e entrypoints no checkout atual. Nunca versione `.env` nem chaves.

1. Visão geral

```
Lead / canal de entrada (ex.: Telegram)
            |
            v
Python AI — localhost:8001
  FastAPI, agentes, estado da conversa,
  prompts, integração LLM e cliente HTTP
            |
            | HTTP + X-Correlation-ID
            v
Go API — localhost:8080
  HTTP handlers/middleware
  Application / casos de uso
  Domain / regras de negócio
  Repositories / Infrastructure / sqlc
            |
            v
PostgreSQL — localhost:5432
```

O Python conversa com o backend Go por HTTP. A persistência de negócio fica concentrada na API Go; o Python não deve acessar diretamente o PostgreSQL de negócio. O `X-Correlation-ID` permite correlacionar logs entre os serviços.

2. Projeto Python --- `real-estate-sdr-ai`

Responsabilidade

Receber/processar mensagens, interpretar intenção, manter o estado da conversa, qualificar critérios (tipo de imóvel, região, orçamento, quartos e urgência), chamar o LLM e consultar a API Go para localizar

imóveis compatíveis.

Estrutura conhecida

```
real-estate-sdr-ai/
■■■ ai/
■   ■■■ app/
■       ■■■ main.py       # aplicação FastAPI
■       ■■■ agent/        # agentes e fluxo conversacional
■       ■■■ api/          # endpoints e rotas
■       ■■■ clients/      # clientes HTTP para serviços externos/API Go
■       ■■■ llm/          # integração com modelo de linguagem
■       ■■■ models/       # modelos de entrada/saída
■       ■■■ prompt/       # prompts
■       ■■■ services/     # serviços de aplicação e lógica de apoio
■■■ requirements.txt      # dependências, se localizado na raiz
■■■ .env.example          # modelo de configuração, se presente
■■■ .env                  # configuração local/secrets; não versionar
■■■ .venv/                # ambiente virtual local; não versionar
```

Confira os arquivos reais com `Get-ChildItem -Recurse .\ai\app`; a lista acima é a organização lógica conhecida, não uma garantia de que todos os diretórios estejam preenchidos.

Fluxo de processamento

1. A API recebe a mensagem e identifica o lead/conversa.
2. O serviço carrega o estado conversacional conhecido.
3. O fluxo de agente/qualificação identifica intenção e critérios.
4. O LLM interpreta a mensagem e/ou ajuda a compor a resposta conforme os prompts configurados.
5. O estado é atualizado e a máquina de estados define a próxima ação.
6. Quando precisa buscar imóveis, Python chama a API Go.
7. O resultado é convertido em recomendação ou resposta sem resultados.
8. A API devolve a resposta ao canal de entrada.

Inicialização no Windows PowerShell

```
cd \codes\fiap\8IADT-tech-challenge-fase-5\real-estate-sdr-ai
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Se `.venv` já existir, não é necessário recriá-lo. Se o PowerShell bloquear a ativação, use o executável diretamente em `.venv\Scripts\python.exe`.

Configure o `.env` com base no `.env.example`, se existir. Confira as variáveis realmente lidas pelo código, incluindo a chave do provedor LLM (por exemplo, `OPENAI_API_KEY`, se aplicável), a chave interna (`AI_SERVICE_API_KEY`, se usada) e a URL da API Go. Não coloque valores secretos no README.

Inicie a API:

```
.\.venv\Scripts\python.exe -m uvicorn ai.app.main:app --host 0.0.0.0 --port 8001 --reload
```

Em outro terminal:

```
Invoke-RestMethod http://localhost:8001/health
```

Se estiver habilitado, a documentação OpenAPI costuma ficar em `http://localhost:8001/docs`; confirme no `ai/app/main.py`.

Testes Python

Na raiz do projeto:

```
.\.venv\Scripts\python.exe -m pytest -v
.\.venv\Scripts\python.exe -m pytest -q
```

Durante o desenvolvimento, a suíte chegou a passar com 68 testes. O número pode mudar; considere o resultado do checkout atual como referência.

3. Projeto Go --- `real-estate-sdr-api`

Responsabilidade

Expor endpoints REST, aplicar regras de negócio, coordenar casos de uso e persistir os dados em PostgreSQL. O projeto contém migrations SQL e consultas organizadas com `sqlc`.

Estrutura conhecida

```
real-estate-sdr-api/
■■■ .env
■■■ .env.example
■■■ docker-compose.yml
■■■ scripts/
■■■ backend/
    ■■■ cmd/              # entrypoints
    ■■■ internal/
    ■   ■■■ application/  # casos de uso/orquestração
    ■   ■■■ domain/       # conceitos e regras de domínio
    ■   ■■■ http/         # handlers, rotas, middleware e integração HTTP
    ■   ■■■ infrastructure/# banco e integrações técnicas
    ■   ■■■ repository/   # acesso a dados
    ■   ■■■ observability/# logging/correlação, conforme implementação
    ■■■ migrations/       # migrations SQL
    ■■■ queries/          # consultas SQL usadas pelo sqlc
    ■■■ Dockerfile
    ■■■ go.mod
    ■■■ sqlc.yaml
```

Confirme a estrutura exata com `Get-ChildItem .\backend\internal -Recurse`.

Camadas

- Domain: regras e conceitos do negócio, sem depender de detalhes de HTTP ou banco.
- Application: coordena casos de uso.
- HTTP/API: recebe requisições, valida dados de transporte e chama os casos de uso.

- Repository/Infrastructure: implementa persistência e integrações externas.
- Queries/sqlc: consultas SQL e código Go tipado gerado.
- Migrations: evolução versionada do schema.

A direção recomendada das dependências é de fora para dentro: handlers/infraestrutura dependem de casos de uso e abstrações; o domínio deve permanecer independente de detalhes técnicos.

Subir infraestrutura com Docker Compose

Na raiz de `real-estate-sdr-api`:

```
cd \codes\fiap\8IADT-tech-challenge-fase-5\real-estate-sdr-api
Get-Content .\docker-compose.yml
docker compose up -d
docker compose ps
```

O ambiente usado durante o desenvolvimento tem PostgreSQL em `real-estate-sdr-postgres`, banco `real_estate_sdr`, porta local `5432`. Use o `docker-compose.yml` atual para confirmar serviço, credenciais, volume e portas.

Se houver `.env.example`, crie o `.env` local a partir dele e preencha os valores necessários. Não compartilhe credenciais.

Migrations

As migrations ficam em `backend/migrations`. O banco usado no desenvolvimento registrou migrations `001` a `007`. A migration 007 adicionou `properties.status`, cujos valores válidos incluem `AVAILABLE`, `RESERVED`, `SOLD`, `RENTED` e `INACTIVE`.

Ter migrations no banco não garante que um banco novo será migrado automaticamente. Verifique no código/scripts se a aplicação aplica migrations no startup ou se há um comando específico. Não apague volumes nem execute alterações destrutivas sem backup.

Executar e testar Go

```
cd .\backend
go mod download
go test -count=1 ./...
```

Para localizar o entrypoint:

```
Get-ChildItem .\cmd -Recurse
```

Execute o pacote que realmente existir em `cmd`. Se o entrypoint for `cmd\api`, por exemplo:

```
go run .\cmd\api
```

A API local foi configurada para a porta `8080`. Valide:

```
Invoke-RestMethod http://localhost:8080/health
Invoke-RestMethod http://localhost:8080/ready
```

Se a aplicação for executada pelo Compose, na raiz do projeto:

```
docker compose up --build -d
docker compose ps
docker compose logs -f
```

Não execute `docker compose` dentro de `backend` se o arquivo estiver somente na raiz.

⚙ 4. Configuração e integração Python ↔ Go

A URL usada pelo Python deve corresponder à topologia em que os serviços estão rodando.

- Ambos no Windows: o Python pode usar `http://localhost:8080` para alcançar a API Go local.
- Go em Docker, Python no Windows: com a porta Go publicada, normalmente use `http://localhost:8080`.
- Python em Docker, Go no host: no Docker Desktop para Windows, use `http://host.docker.internal:8080` se a porta do host estiver publicada.
- Ambos no Compose: use o nome DNS do serviço Go e a porta interna do container; `localhost` dentro de um container aponta para o próprio container.
- Azure/produção: não use `host.docker.internal`; configure a URL pública/privada correta e proteja a comunicação.

No ambiente de desenvolvimento são usadas `API_KEY`/`X-API-Key` para chamadas protegidas da API Go e, quando habilitado, `AI_SERVICE_API_KEY`/`X-Internal-API-Key` para o serviço de IA. Confira os middlewares e clientes para saber quais endpoints exigem autenticação.

5. Ordem recomendada para iniciar tudo

Abra terminais separados.

Terminal 1 --- banco e infraestrutura

```
cd \codes\fiap\8IADT-tech-challenge-fase-5\real-estate-sdr-api
docker compose up -d
docker compose ps
```

Confirme que o PostgreSQL está saudável e que as migrations necessárias foram aplicadas.

Terminal 2 --- API Go

Se o Compose já iniciar a API, acompanhe os logs:

```
docker compose logs -f
```

Se for executar diretamente, entre em `backend`, confira `cmd` e use o entrypoint correto. Depois valide `/health` e `/ready` na porta `8080`.

Terminal 3 --- Python AI

```
cd \codes\fiap\8IADT-tech-challenge-fase-5\real-estate-sdr-ai
.\.venv\Scripts\python.exe -m uvicorn ai.app.main:app --host 0.0.0.0 --port 8001 --reload
```

Valide `/health` na porta `8001`.

Terminal 4 --- Telegram

O script conhecido é `ai/scripts/telegram_bot.py`; confirme o caminho atual e as variáveis necessárias antes de iniciar:

```
cd \codes\fiap\8IADT-tech-challenge-fase-5\real-estate-sdr-ai
.\.venv\Scripts\python.exe .\ai\scripts\telegram_bot.py
```

O bot pode precisar de token Telegram e da URL/chave da API Go. Não exponha esses valores em logs ou no Git.

6. Diagnóstico rápido

```
docker compose ps
docker compose logs --tail 100
Test-NetConnection localhost -Port 5432
Test-NetConnection localhost -Port 8080
Test-NetConnection localhost -Port 8001
Invoke-RestMethod http://localhost:8001/health
Invoke-RestMethod http://localhost:8080/health
Invoke-RestMethod http://localhost:8080/ready
```

Problemas frequentes:

- Python não alcança Go: confirme a variável da URL base e a topologia host/container.
- 401/403: confira `X-API-Key` e/ou `X-Internal-API-Key`.
- Banco indisponível: confira logs do PostgreSQL, credenciais, porta e configuração do Go.
- Tabela/coluna inexistente: confira se as migrations foram aplicadas ao banco correto.
- Dependência Python ausente: instale `requirements.txt` no `.venv` usado para iniciar a API.
- Falha em testes: execute a partir da raiz correta e confira dependências externas exigidas.

7. Segurança e boas práticas

1. Não versione `.env`, tokens ou chaves.
2. Mantenha `.env.example` sem secrets reais.
3. Não exponha PostgreSQL diretamente à internet.
4. Proteja endpoints e valide entrada.
5. Propague `X-Correlation-ID` e inclua-o nos logs.
6. Configure timeouts e tratamento de falhas nos clientes HTTP.
7. Centralize o acesso ao banco de negócio na API Go.
8. Em produção, remova `--reload` e configure health checks, logs, recursos e secrets.
9. Versione migrations e valide compatibilidade do schema antes do deploy.
10. Faça backup antes de operações destrutivas.

8. Checklist de inicialização

- [ ] PostgreSQL iniciou sem erros.
- [ ] Migrations necessárias estão aplicadas ao banco correto.
- [ ] API Go iniciou; `/health` responde.
- [ ] `/ready` confirma a prontidão do backend/banco.
- [ ] `go test -count=1 ./...` passa.
- [ ] `.venv` e dependências Python estão corretos.
- [ ] Variáveis de ambiente do Python estão configuradas.
- [ ] Python iniciou; `/health` responde.
- [ ] Python alcança a API Go pela URL correta.
- [ ] `python -m pytest -q` passa.
- [ ] Uma requisição de ponta a ponta foi testada com lead de teste.
- [ ] Logs permitem correlacionar a requisição entre Python e Go.

9. Itens para conferir no checkout atual

Antes de usar este documento como procedimento de produção, confirme:

- caminho exato do entrypoint Go em `backend/cmd`;
- nomes das variáveis de ambiente realmente lidas por cada serviço;
- se o Compose aplica migrations automaticamente;
- nomes dos serviços, portas e health checks no `docker-compose.yml`;
- caminho real da suíte de testes Python;
- rotas HTTP e requisitos de autenticação;
- recursos adicionais necessários ao empacotamento Docker do Python.

Este README descreve a arquitetura conhecida durante o desenvolvimento e evita tratar configurações locais específicas como garantidas em todos os ambientes.

10. Segurança

Incluí também os principais controles a documentar e validar:

- Autenticação entre serviços: X-API-Key e X-Internal-API-Key, conforme as rotas configuradas.
- Proteção do banco: PostgreSQL sem exposição pública e privilégios mínimos.
- Segurança de dependências: análise de vulnerabilidades em pacotes e imagens Docker.
- Proteção de dados: evitar chaves, tokens e dados pessoais desnecessários nos logs, uso de arquivos .env.
- Resiliência: timeouts, validação de entrada e limites de requisições.

11. Observabilidade

- Tracing distribuído com OpenTelemetry.
- X-Correlation-ID, Já identificado no fluxo para correlacionar requisições entre Python e Go.

12. Tecnologias

Tecnologia	Finalidade
Python 3.11	Linguagem principal
FastAPI	Framework para API HTTP
Uvicorn	Servidor ASGI
pytest	Testes automatizados
LLM / OpenAI	Interpretação e geração de respostas, conforme configuração
HTTP Client	Comunicação com a API Go
Docker	Containerização, quando configurada

> Confira o `requirements.txt` para validar as bibliotecas e versões efetivamente utilizadas.

13 Fluxo de atendimento

```
flowchart TD
    A["Mensagem do lead"] --> B["Classificação da intenção"]
    B --> C["Extração dos critérios"]
    C --> D["Atualização do estado"]
    D --> E{"Critérios suficientes?"}
    E -- "Não" --> F["Perguntar o próximo critério"]
    F --> G["Responder e aguardar"]
    G --> A
    E -- "Sim" --> H["Consultar API Go"]
    H --> I{"Imóveis encontrados?"}
    I -- "Sim" --> J["Preparar recomendação"]
    I -- "Não" --> K["Informar ausência de opções"]
    J --> L["Gerar resposta"]
    K --> L
    L --> M["Responder ao lead"]
```

O fluxo representa o comportamento esperado da solução. As transições devem ser conferidas na implementação atual da máquina de estados.

```
## 14. ■ Componentes principais

| Componente | Responsabilidade |
|---|---|
| FastAPI | Exposição dos endpoints HTTP |
| Agent Router | Direcionamento do fluxo conforme a intenção |
| Qualification Agent | Extração e qualificação dos critérios do lead |
| State Machine | Controle das etapas da conversa |
| Property Agent | Coordenação da busca e recomendação de imóveis |
| LLM Client | Integração com o modelo de linguagem |
```

| Go API Client | Comunicação com o backend de negócio |
| Property Matching Service | Aplicação dos critérios de compatibilidade |

> Os nomes e as responsabilidades devem ser ajustados caso a implementação atual utilize outras classes ou módulos.

15. ■ Testes automatizados

Execute os testes a partir da raiz do projeto:

```
.\.venv\Scripts\python.exe -m pytest -v
```

Para uma execução resumida:

```
.\.venv\Scripts\python.exe -m pytest -q
```

Os testes devem cobrir a extração de critérios, as mudanças de estado, a busca de imóveis, os casos sem resultados e o tratamento de falhas nas integrações.

16. ■ Observabilidade

O serviço deve permitir acompanhar as requisições, falhas e latência das integrações.

- Logs estruturados.
- Propagação do `X-Correlation-ID`, conforme a implementação.
- Endpoint de saúde `/health`.
- Monitoramento das chamadas ao LLM e à API Go.

Evolução recomendada

- OpenTelemetry para instrumentação de traces.
- Prometheus para coleta de métricas.
- Grafana para dashboards.
- Loki ou Azure Monitor para centralização de logs.

Essas ferramentas são recomendações e não devem ser consideradas implementadas sem validação no projeto.

■ Segurança

- Manter chaves e credenciais fora do Git.
- Utilizar variáveis de ambiente para configurações sensíveis.
- Validar mensagens e parâmetros recebidos.
- Configurar timeouts nas chamadas externas.
- Evitar registrar tokens ou dados pessoais desnecessários.
- Utilizar HTTPS em ambientes de produção.
- Tratar o conteúdo enviado ao LLM como entrada não confiável.